
SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 — ARCHITECTURE DESIGN ASSIGNMENT

Project Title:
AI-Powered Smart Volunteer and NGO Resource Management Platform

Course Code	CSA1017
Course Title	Software Engineering
Assessment Tool	Assessment Tool 2 — Architecture Design Assignment
Student Name	R. Indhu
Register No.	192524332
Department	AI & DS
Academic Year	2025 - 2029

Code of Conduct

I R. Indhu (192524332) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R. Indhu

Table of Contents

1. Introduction and Industry Context	4
1.1 Problem Statement	4
1.2 Objectives.....	4
1.3 Expected Outcomes.....	4
2. System Requirements Analysis.....	5
2.1 Functional Requirements	5
2.2 Non-Functional Requirements and Quality Attributes	5
2.3 Stakeholder Analysis.....	6
3. Selection of Suitable Software Architecture.....	7
3.1 Candidate Architectural Styles Considered	7
3.2 Selected Architecture	7
3.3 Justification for the Selected Architecture	8
4. Software Architecture Diagram	9
4.1 Layer Overview.....	9
5. Explanation of Components and Their Interactions	10
5.1 Component Responsibilities	10
5.2 Communication and Data Exchange.....	11
5.3 Overall System Workflow	11
6. Discussion of Quality Attributes.....	11
7. Justification of Architectural Decisions.....	12
7.1 Rationale Behind Key Decisions	12
7.2 Trade-offs Considered.....	13
7.3 Support for Future Enhancements and Long-Term Maintainability	13
8. Proposed Technology Stack.....	14
9. Implementation Roadmap and Testing Strategy.....	14
9.1 Phased Implementation Plan.....	14
9.2 Testing Strategy	15
10. Conclusion.....	15

1. Introduction and Industry Context

Non-Governmental Organizations (NGOs) and community volunteer networks form the backbone of humanitarian service delivery across education, healthcare, disaster relief, and community development programs. Their effectiveness depends heavily on the ability to mobilize the right volunteers for the right tasks, allocate scarce resources efficiently, and maintain transparency with donors and beneficiaries.

Advances in Artificial Intelligence (AI), cloud computing, Geographic Information Systems (GIS), and mobile technologies now make it possible to build intelligent platforms that can match volunteers to suitable activities automatically, monitor project outcomes in real time, and provide administrators with data-driven dashboards for decision-making.

1.1 Problem Statement

NGOs often face difficulties in managing volunteers, assigning tasks, tracking donations, and monitoring project outcomes due to fragmented systems. Manual coordination leads to delays and inefficient resource utilization. Volunteer skills and availability are rarely matched systematically against actual project needs, donations are tracked in isolated spreadsheets with little audit trail, and administrators lack a consolidated, real-time view of ongoing projects.

1.2 Objectives

1. Manage volunteer registrations digitally through a unified onboarding platform.
2. Match volunteers to appropriate tasks and projects using AI-driven recommendation techniques.
3. Monitor NGO project progress with real-time status tracking and milestone reporting.
4. Track donations and resources with complete transparency and auditability.
5. Optimize volunteer allocation across concurrent projects and geographies.

1.3 Expected Outcomes

- Improved volunteer participation through personalized, relevant task recommendations.
- Efficient resource allocation that minimizes idle capacity and duplication of effort.
- Better project management with continuous visibility into progress and risks.
- Enhanced organizational transparency across all operational layers.

-
- Increased donor confidence through verifiable donation tracking and impact reporting.

2. System Requirements Analysis

A structured requirements analysis is essential to translate the problem statement into concrete architectural drivers. Requirements are grouped into functional requirements (what the system must do) and non-functional requirements (the quality attributes that constrain how well it must do it). Both categories directly shape the choice of architectural style described in Section 3.

2.1 Functional Requirements

ID	Functional Requirement
FR-1	The system shall allow volunteers to register, create profiles, and specify skills, interests, location, and availability.
FR-2	The system shall allow NGOs to create, publish, and manage projects, tasks, and campaigns.
FR-3	The AI Matching Engine shall recommend suitable volunteers for a given task based on skills, proximity, and availability.
FR-4	The system shall allow donors to make monetary or in-kind donations and track their utilization.
FR-5	The system shall track task assignment, completion status, and volunteer hours contributed.
FR-6	The system shall provide real-time dashboards for administrators showing project and resource status.

2.2 Non-Functional Requirements and Quality Attributes

The following non-functional requirements were identified as the key quality attributes influencing the software architecture. Scalability and availability are particularly critical because volunteer demand spikes sharply during disaster-relief events, while security is critical because the platform handles personally identifiable information (PII) and financial transactions.

Quality Attribute	Description / Requirement
Scalability	The platform must support a growing number of concurrent volunteers, NGOs, and projects without performance degradation, scaling horizontally during disaster-response surges.
Security	Volunteer and donor personal data, payment information, and organizational records must be protected using encryption, authentication, and role-based access control.
Availability	The system should maintain high uptime (target 99.9%) since disaster-relief coordination cannot tolerate prolonged outages.
Performance	AI matching and dashboard queries should return results within 2–3 seconds under normal load to keep the platform responsive.
Reliability	The system must process donation and task-assignment transactions consistently and recover gracefully from partial failures.

2.3 Stakeholder Analysis

- Volunteers — register, discover, and complete tasks; need simple mobile access and timely notifications.
- Donors / Sponsors — contribute funds or resources; need transparency and impact visibility.
- NGO Administrators / Coordinators — manage projects, allocate resources, and monitor outcomes.

3. Selection of Suitable Software Architecture

3.1 Candidate Architectural Styles Considered

Several architectural styles were evaluated against the requirements identified in Section 2 before finalizing the design.

Architectural Style	Suitability Assessment
Monolithic Layered Architecture	Simple to build initially, but scaling individual features (e.g., AI matching during a disaster surge) independently is not possible; a single point of failure risks the entire platform.
Client-Server	Adequate for small deployments but does not natively support asynchronous, real-time updates across many concurrent NGOs and volunteers.
Pure Microservices (synchronous only)	Provides modularity and independent scaling, but tight service-to-service coupling via synchronous REST calls creates cascading failures under load.
Event-Driven Architecture (pure)	Excellent for asynchronous, real-time processing (notifications, matching), but lacks the clear service boundaries needed for independent deployment and team ownership.

3.2 Selected Architecture

The proposed system adopts a Hybrid Microservices + Event-Driven Architecture (EDA), fronted by an API Gateway and supported by a dedicated AI/ML and GIS intelligence layer. Each core capability of the platform — volunteer management, AI matching, task allocation, donation and resource tracking, monitoring and reporting, and notifications — is implemented as an independently deployable microservice.

3.3 Justification for the Selected Architecture

- **Independent Scalability:** During disaster-relief surges, the AI Matching Service and Notification Service can be scaled independently of, say, the Reporting Service, optimizing cloud cost and responsiveness.
- **Loose Coupling via Events:** Publishing events such as VolunteerProfileUpdated or TaskMatched allows services to react asynchronously without blocking one another, improving resilience and responsiveness.
- **Fault Isolation:** A failure in one microservice (e.g., the analytics engine) does not bring down volunteer registration or donation processing.

- **Technology Flexibility:** The AI/ML matching service can use Python-based ML frameworks while other services use Node.js or Java, each choosing the best-fit technology stack.

4. Software Architecture Diagram

The diagram below illustrates the overall structure of the proposed platform, organized into six logical layers: the Presentation Layer, the API Gateway & Security Layer, the Application/Microservices Layer, the Event Bus, the AI/ML & GIS Intelligence Layer, and the Data Persistence Layer, supported by External Systems and Cloud Infrastructure.

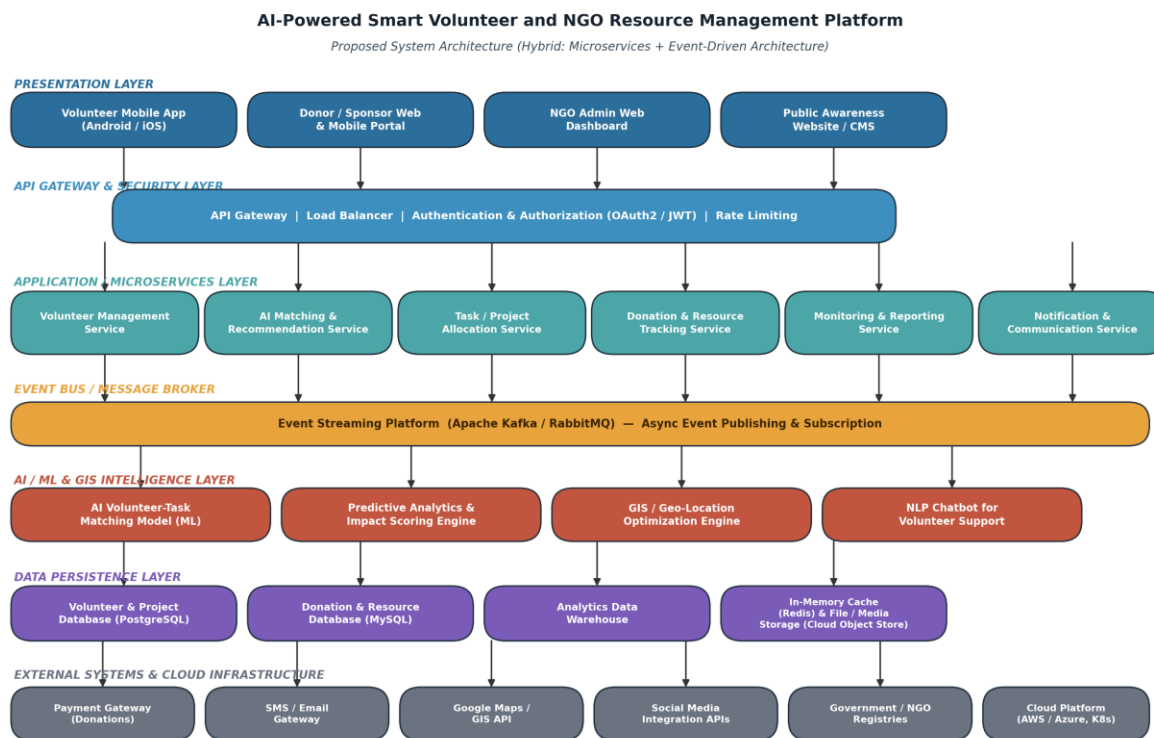


Figure 1. Layered Hybrid Microservices + Event-Driven Architecture of the AI-Powered Smart Volunteer and NGO Resource Management Platform.

4.1 Layer Overview

Layer	Purpose
Presentation Layer	Volunteer mobile app, donor/sponsor portal, NGO admin dashboard, and public awareness website — the entry points for all user interaction.

Layer	Purpose
API Gateway & Security Layer	Single entry point for all client requests; performs authentication, authorization, rate limiting, and load balancing.
Application / Microservices Layer	Independently deployable services implementing core business capabilities: volunteer management, AI matching, task allocation, donation tracking, monitoring/reporting, and notifications.
Event Bus / Message Broker	Asynchronous publish-subscribe backbone (Kafka/RabbitMQ) decoupling services and enabling real-time event propagation.
AI/ML & GIS Intelligence Layer	Machine-learning matching model, predictive analytics/impact scoring, GIS-based geo-optimization, and an NLP chatbot for volunteer support.

5. Explanation of Components and Their Interactions

5.1 Component Responsibilities

Component	Purpose and Responsibilities
Volunteer Management Service	Handles volunteer registration, profile management, skills/availability data, and authentication data hand-off to the API Gateway.
AI Matching & Recommendation Service	Applies machine-learning models to match volunteer skills/location/availability against NGO project and task requirements.
Task / Project Allocation Service	Manages project and task lifecycle: creation, publishing, assignment, and completion tracking.
Donation & Resource Tracking Service	Processes monetary/in-kind donations, integrates with payment gateways, and maintains an auditable resource ledger.

Component	Purpose and Responsibilities
Monitoring & Reporting Service	Aggregates task, donation, and project data to generate real-time dashboards and periodic organizational reports.

5.2 Communication and Data Exchange

Client applications communicate with the backend exclusively through the API Gateway using secure REST/HTTPS calls, which handles authentication and routes requests to the appropriate microservice. Services that need to react to state changes elsewhere in the system — for example, the AI Matching Service reacting to a new volunteer profile — subscribe to relevant topics on the event bus rather than calling one another directly.

The sequence diagram below illustrates a representative interaction: a volunteer updates their profile, the AI Matching Service is triggered asynchronously via the event bus, a suitable task match is found, and the volunteer is notified — while the Monitoring & Reporting Service is updated in parallel.

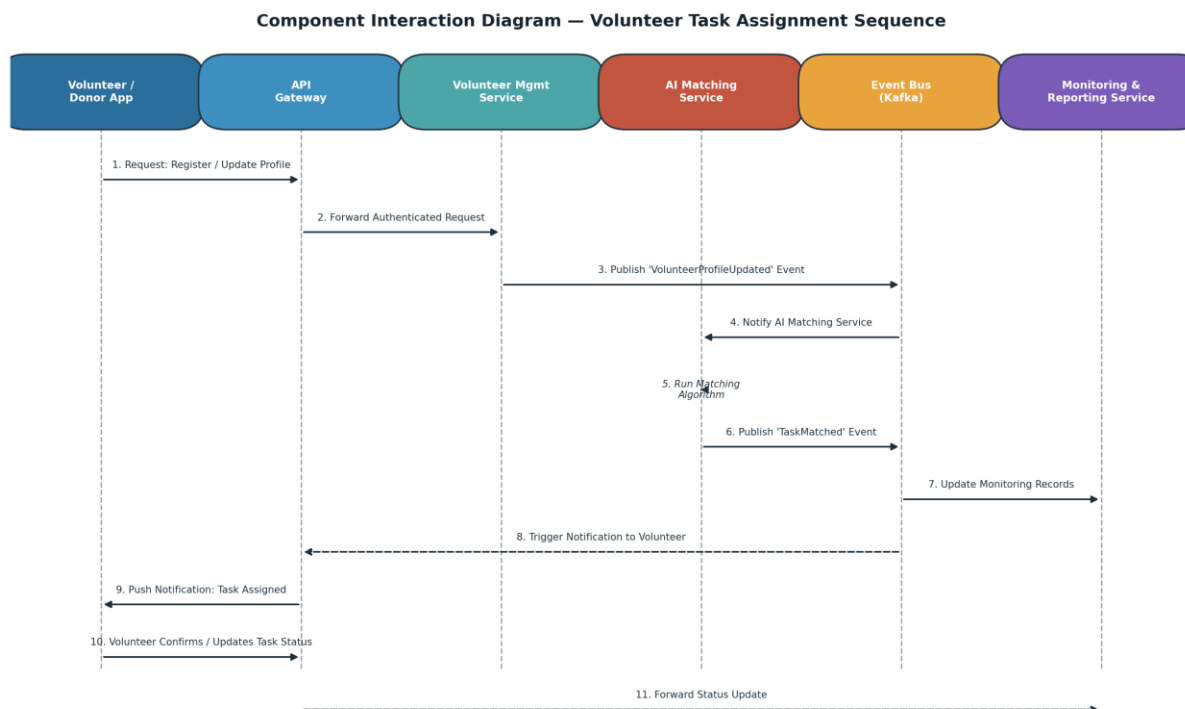


Figure 2. Component Interaction Diagram showing the volunteer task-assignment sequence across services and the event bus.

5.3 Overall System Workflow

The end-to-end workflow — from volunteer registration through AI-driven task matching to progress monitoring and reporting — is illustrated in the flowchart below. This workflow underpins the primary value proposition of the platform: reducing the time and manual effort needed to connect a volunteer's skills to an NGO's actual need.

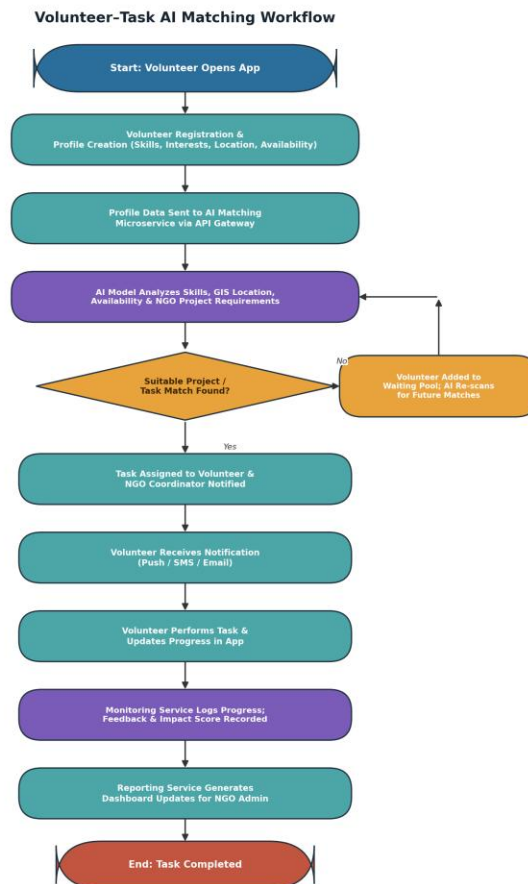


Figure 3. Volunteer–Task AI Matching Workflow.

6. Discussion of Quality Attributes

This section evaluates how the proposed hybrid microservices and event-driven architecture addresses each of the key quality attributes identified during requirements analysis. The radar chart below provides a qualitative rating (on a 1–10 scale) of how strongly the architecture satisfies each attribute, followed by a detailed discussion.

**Quality Attribute Rating of Proposed Architecture
(Scale 1-10)**

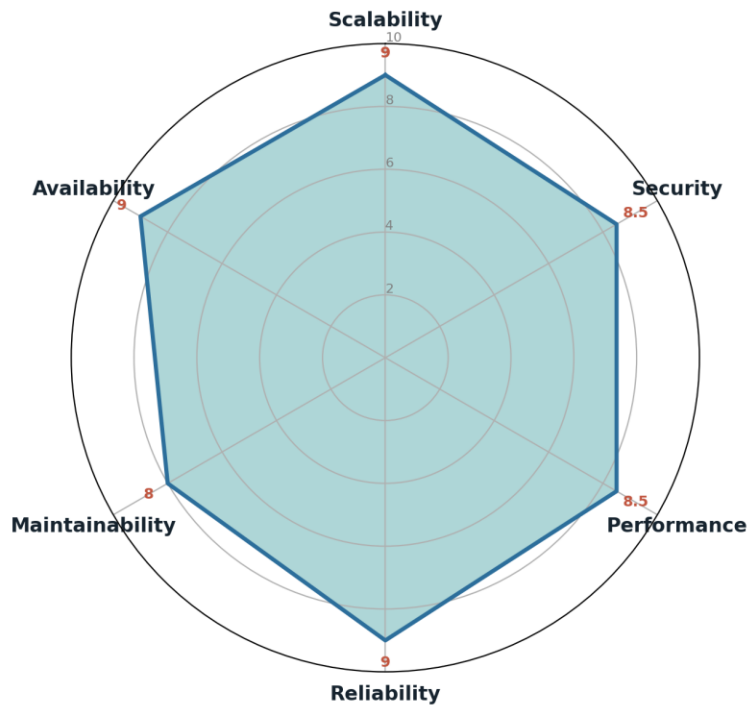


Figure 4. Qualitative rating of the proposed architecture against the six key quality attributes.

Quality Attribute	How the Architecture Addresses It
Scalability	The microservices layer scales horizontally behind the API Gateway/load balancer; the event bus buffers load spikes (e.g., during disaster-relief surges) so that individual services are not overwhelmed. Container orchestration (Kubernetes) enables auto-scaling based on demand.
Security	OAuth2/JWT-based authentication and role-based access control are enforced at the API Gateway; sensitive donor and volunteer data is encrypted in transit (TLS) and at rest; the payment gateway integration is isolated within the Donation & Resource Tracking Service to minimize the compliance surface (e.g., PCI-DSS).

Quality Attribute	How the Architecture Addresses It
Performance	The event-driven design avoids blocking synchronous chains; frequently accessed data (volunteer/task listings) is served from the Redis cache; the AI matching model is deployed as a dedicated service so heavy computation does not degrade the responsiveness of other services.
Reliability	Asynchronous messaging with the event bus provides at-least-once delivery and replay capability, so a temporarily unavailable service does not lose events; database transactions ensure donation records remain consistent.

7. Justification of Architectural Decisions

7.1 Rationale Behind Key Decisions

The decision to combine microservices with an event-driven backbone, rather than choosing either style in isolation, was driven by the dual nature of the platform's requirements: it needs the independent scalability and deployability of microservices, and the real-time, loosely-coupled responsiveness of event-driven systems.

This isolates computationally expensive matching and analytics workloads (which may require GPU resources or specialized ML runtimes) from lightweight, latency-sensitive transactional services such as task allocation, allowing each to be scaled and optimized on its own terms.

7.2 Trade-offs Considered

Trade-off	Discussion
Complexity vs. Modularity	Microservices introduce operational complexity (service discovery, distributed tracing, deployment pipelines) compared to a monolith. This was accepted because the long-term benefits of independent scaling and fault isolation outweigh the added DevOps overhead, especially as the platform grows to serve many NGOs.

Trade-off	Discussion
Eventual Consistency vs. Strong Consistency	The event-driven design favors eventual consistency between services (e.g., a brief delay before the dashboard reflects a new donation). This is acceptable for reporting and matching use cases, while donation transactions themselves are kept strongly consistent within the Donation & Resource Tracking Service.
Build vs. Buy (AI Model)	A custom-trained matching model offers better fit to NGO-specific data than an off-the-shelf recommendation engine, at the cost of higher initial development effort; this was judged worthwhile given matching quality is central to the platform's value proposition.

7.3 Support for Future Enhancements and Long-Term Maintainability

- New capabilities (e.g., a volunteer skill-certification service, or a disaster-prediction module) can be added as new microservices that subscribe to existing events, without modifying existing services.
- The AI Matching Service can be retrained or replaced with an improved model version independently, using a blue-green or canary deployment strategy.
- The event bus provides a natural integration point for future third-party systems (e.g., a national disaster-management platform) to consume platform events.

8. Proposed Technology Stack

The following technology stack was selected to realize the architecture described in Sections 3–5. Choices favor mature, well-supported, open-source technologies to minimize licensing cost for NGOs, while retaining the flexibility to deploy on any major cloud provider.

Layer / Concern	Recommended Technologies
Presentation Layer	React Native / Flutter (mobile apps), React.js (web portal and admin dashboard), Progressive Web App support for low-bandwidth regions.

Layer / Concern	Recommended Technologies
API Gateway	Kong / AWS API Gateway with OAuth2 and JSON Web Tokens (JWT) for authentication and authorization.
Microservices	Node.js / Java Spring Boot for transactional services (Volunteer Management, Task Allocation, Donation Tracking); Python (FastAPI) for the AI Matching Service.
Event Bus	Apache Kafka (preferred for high-throughput event streaming) or RabbitMQ (for simpler deployments).
AI / ML	Scikit-learn / TensorFlow for the matching and recommendation model; spaCy or a transformer-based model for the NLP support chatbot.
GIS	Google Maps Platform / OpenStreetMap with geospatial indexing for volunteer-to-task proximity optimization.
Databases	PostgreSQL (volunteer/project data), MySQL (donation/resource ledger), Snowflake/BigQuery (analytics warehouse), Redis (caching).

9. Implementation Roadmap and Testing Strategy

9.1 Phased Implementation Plan

Phase	Key Activities
Phase 1 — Foundation (Months 1–2)	Set up cloud infrastructure, API Gateway, authentication, and the Volunteer Management and Task Allocation microservices; build the volunteer mobile app MVP.
Phase 2 — Intelligence Layer (Months 3–4)	Develop and train the AI Matching model on historical/sample data; integrate the event bus; add GIS-based location optimization.
Phase 3 — Donations & Reporting (Months 5–6)	Build the Donation & Resource Tracking Service with payment gateway integration.

Phase	Key Activities
Phase 4 — Pilot Deployment (Month 7)	Deploy to a pilot set of partner NGOs; gather feedback on matching accuracy and usability; monitor system performance under real load.
Phase 5 — Scale-Out (Months 8+)	Onboard additional NGOs and regions; enable auto-scaling policies; expand the notification channels and add the NLP support chatbot.

9.2 Testing Strategy

- **Unit Testing:** Each microservice is unit-tested in isolation, with the AI Matching Service additionally validated against labeled historical matching data.
- **Integration Testing:** Contract tests verify event schemas published/consumed across the event bus; API Gateway routing is tested end-to-end.
- **Load Testing:** Simulated disaster-surge scenarios validate that the Notification and AI Matching Services auto-scale correctly under sudden spikes in volunteer sign-ups.

10. Conclusion

This report analyzed the requirements of an AI-Powered Smart Volunteer and NGO Resource Management Platform and proposed a Hybrid Microservices and Event-Driven Architecture as the most suitable design. The architecture directly addresses the core problem of fragmented, manual NGO coordination by providing AI-driven volunteer matching, real-time project monitoring, transparent donation tracking, and consolidated administrative dashboards.

The layered design — spanning presentation, API gateway, microservices, event bus, AI/ML intelligence, and data persistence — ensures that the platform can scale to meet surges in demand during disaster-relief operations, remain secure and reliable for handling sensitive donor and volunteer data, and stay maintainable as new features and NGO partners are added over time. By meeting the identified functional and non-functional requirements, the proposed architecture is well positioned to improve volunteer participation, resource utilization, organizational transparency, and ultimately, social impact.